Informatik 1: KI-Tools in der Programmierung

Wirtschaftsingenieurwesen Bachelor

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Generative KI und Lernen
2. KI als Lernpartner: Lernmodi und gute Fragen
3. KI-Tools: Kategorien und Empfehlungen
4. Typische KI-Fehler und warum sie entstehen
5. Strategien zur Fehlerreduktion

TL;DL – Kurzfassung

- **KI und Lernen:** Erklärhilfe nutzen, nicht Lösungsgenerator
- **KI als Lernpartner:** Lernmodus an, Kontext geben, Dialog statt fertiger Lösung
- **KI-Tools:** Chat, Notebook-Assistent, Completion, Agenten – und was davon jetzt sinnvoll ist
- **KI-Fehler:** 62% unsicherer Code – KI-Output nie automatisch korrekt
- **Fehlerreduktion:** Reviewen, testen, erklären lassen

1. Generative KI-Tools: Chancen

Studie Dell’Acqua et al. (2023, Harvard Business School)

- 750 Berater im Beratungskontext untersucht
- **Bis zu 25% schneller** (Produktivitätssteigerung)
- **Bis zu 40% besser** (Qualitätssteigerung)
- Durchschnittliche Berater profitieren besonders (43% vs 17%)

1. Generative KI-Tools: Verbreitung

Stack Overflow Developer Survey 2025

- **84%** der Entwickler nutzen oder planen KI-Tools (2024: 76%)
- **51%** nutzen KI-Tools täglich
- Aber: Nur **29%** vertrauen der Genauigkeit von KI-Output (2024: 40%)

Hohe Nutzung, geringes Vertrauen – ein Widerspruch, der zum Nachdenken anregen sollte.

1. Generative KI-Tools: Forschung

LLMs im Requirements Engineering (SIENA, Zwanzig/Dietrich 2023)

- GPT-3.5 und GPT-4 zur Extraktion von Nutzeranforderungen
- Fallstudie in der Physiotherapie
- Sensitivität: GPT-3.5 ca. 71%, GPT-4 ca. 73%
- Modelle identifizierten sogar neue, vorher unbekannte Anforderungen

1. Effekte auf den Lernerfolg: Ergebnisse

Ergebnisse aus 10 Studien

1. **Erklärende Nutzung** (Konzepte erklären lassen): positive Lerneffekte (+12,5 Punkte, +6,8% Examensanstieg)
2. **Lösungsorientierte Nutzung** (Codegenerierung): teilweise negative Effekte
3. Effekte hängen von Promptqualität und didaktischem Kontext ab

Gezielter Einsatz wirkt – **häufigerer** Einsatz nicht (El Fathi et al. 2025, n=120).

1. Effekte auf den Lernerfolg: Positiv

Meta-Analyse Alanazi et al. (2025)

- 35 kontrollierte Studien ausgewertet
- Signifikante Leistungsverbesserung (SMD = 0,86)
- Schnellere Aufgabenbearbeitung (SMD = -0,69)

Leistungsstarke Lernende hinterfragen KI-Ergebnisse aktiv – **schwächere** akzeptieren sie passiv.

1. Effekte auf den Lernerfolg: Warnungen

Langfristige Risiken

Li et al. (2025): GenKI verbessert kurzfristige Ergebnisse, **beeinträchtigt aber den langfristigen Wissenstransfer**. “Kognitives Outsourcing” verhindert effektives Lernen.

Yang et al. (2025): Studie mit 153 Schülern – ChatGPT-Nutzung führte zu **niedrigerem** Selbstwirksamkeitsempfinden und **schlechteren** Lernergebnissen.

Aufgabe: KI im Studium

Diskutiert in der Gruppe oder mit ChatGPT/Gemini/Claude:

Sollte man ChatGPT zum Lernen im Grundlagenmodul Informatik einsetzen?

- Was spricht dafür?
- Was spricht dagegen?
- Gibt es ein sinnvolles “Wie?”

2. Was sich beim Prompten geändert hat

Weniger Zauberformeln, mehr Kontext

- Aktuelle Modelle **denken selbst schrittweise** (Reasoning) – Tricks wie “Denke Schritt für Schritt” bringen kaum noch etwas
- Die großen Tools haben **eingebaute Lernmodi** – Tutor-Verhalten ist ein Schalter, kein Spezialprompt
- **Dauerhafte Anweisungen** ersetzen das ständige Wiederholen
- Entscheidend ist der **Kontext**: Was wollen Sie, was haben Sie versucht, wo hängen Sie?

Gutes Prompten heißt heute: der KI sagen, **wer Sie sind und was Sie lernen wollen**.

2. Warum Tutor statt Lösung?

KI soll Fragen stellen, nicht Antworten geben

- **Reverse Prompting (Chang et al. 2023):** KI stellt Rückfragen statt direkter Antworten – fördert Planung und Reflexion
- **SocraticAI (Sunil et al. 2025):** >75% der Studierenden produzierten substantielle Reflexionen
- **Sakshm AI (Gupta et al. 2025):** Positiver Einfluss auf Problemlöseverhalten (1.170 Teilnehmer)

Genau dieses Prinzip steckt heute in den **Lernmodi** der großen Tools.

2. Lernmodi nutzen

Tool	Lernmodus	Wo einschalten
ChatGPT	Study Mode	Tools-Menü im Eingabefeld
Gemini	Guided Learning	Tools-Menü im Eingabefeld
Claude	Stil “Learning”	Stil-Auswahl im Eingabefeld
Google Colab	Learn Mode	Gemini-Bereich im Notebook

- Bezeichnungen und Verfügbarkeit ändern sich häufig (Stand Oktober 2026)
- Der Lernmodus erklärt Schritt für Schritt und stellt Rückfragen, statt fertigen Code zu liefern

Für die Übungen in diesem Kurs: Lernmodus an.

2. Dauerhafte Anweisungen

Einmal einstellen, gilt für jeden Chat

In den Einstellungen von ChatGPT, Gemini oder Claude (Personalisierung bzw. Custom Instructions):

> “Ich studiere Wirtschaftsingenieurwesen im 1. Semester und lerne Python. Gib mir keine fertigen Lösungen für Übungsaufgaben. Erkläre Konzepte mit Alltagsbeispielen, stelle Rückfragen und lass mich den nächsten Schritt selbst finden.”

2. Kontext statt Zauberformel

Eine gute Frage enthält

1. **Ziel:** Was soll Ihr Programm tun?
2. **Code:** Ihren eigenen Code, vollständig
3. **Fehlermeldung:** komplett kopiert, nicht abgetippt
4. **Versuch:** Was haben Sie schon probiert?
5. **Stand:** Was verstehen Sie, was nicht?

> Je mehr die KI über **Ihren** Stand weiß, desto besser passt die Erklärung.

2. Schlechte vs. gute Fragen

Schlecht	Besser
“Schreib mir Bubble Sort”	“Ich verstehe nicht, warum Bubble Sort zwei Schleifen braucht. Erkläre es mir an [5, 3, 1]”
“Löse Aufgabe 3”	“Meine Schleife läuft endlos, hier ist mein Code. Stell mir Fragen, bis ich den Fehler selbst finde”
“Korrigiere meinen Code”	“Ich bekomme einen IndexError in Zeile 7. Was bedeutet der Fehler? Zeig mir nicht die Lösung”

2. Dialog statt Einzelprompt

Lernen ist ein Gespräch

1. **Klein anfangen:** eine Frage, ein Konzept
2. **Nachfragen**, bis Sie es in eigenen Worten erklären können
3. **Selbst probieren** und das Ergebnis zurückspielen: “Ich habe es so gelöst – stimmt mein Gedanke?”
4. **Sich abfragen lassen:** “Stell mir 3 Verständnisfragen zu Listen, steigende Schwierigkeit”
5. **Abschließen:** “Fasse zusammen, was ich heute gelernt habe”

2. Dos

Geeignete Nutzung

- “Erkläre mir, was eine for-Schleife macht”
- “Warum bekomme ich einen IndexError?”
- “Stell mir Fragen zu meinem Verständnis von Listen”
- “Reviewe meinen Code und sag mir, was ich verbessern kann – ohne ihn umzuschreiben”

Faustregel: Wenn Sie den KI-Output nicht Zeile für Zeile erklären können, haben Sie nichts gelernt.

2. Don’ts

Ungeeignete Nutzung

- “Schreib mir die Lösung für Übungsaufgabe 4”
- Code kopieren ohne ihn zu verstehen
- KI-Output blind in die Abgabe übernehmen
- **Agenten** (Copilot Agent Mode, Claude Code, Codex) die Übung erledigen lassen

59% der Entwickler nutzen KI-Code, den sie nicht vollständig verstehen (Clutch.co).

2. Überabhängigkeit vermeiden

Übermäßige KI-Nutzung schränkt Problemlösefähigkeit ein

- Wer immer KI fragt, hört auf, **selbst** nach Lösungen zu suchen (Parsakia 2023)
- Positive Effekte bei **gezielter** Nutzung, negative bei Ersatzfunktion
- **Erosion kritischen Denkens** ist das meistgenannte Risiko in der gesamten Literatur (Schei et al. 2024; Vargas-Murillo 2023)

Ziel: KI als Werkzeug beherrschen – nicht von ihr beherrscht werden.

3. KI-Tools: Vier Kategorien

Kategorie	Beispiele	Was es tut	Jetzt sinnvoll?
Chat mit Lernmodus	ChatGPT, Gemini, Claude	erklärt, fragt zurück	**Ja**
Assistent im Notebook	Gemini in Colab	hilft direkt am Code	**Ja, im Learn Mode**
Code-Completion	GitHub Copilot in VS Code	schreibt beim Tippen mit	erst ab Semester 2
Agenten	Copilot Agent Mode, Claude Code, Codex	planen, schreiben, testen selbstständig	**Nein**

Alle Tools basieren auf LLMs – sie **generieren** plausiblen Code, sie **verstehen** ihn nicht.

3. Gemini in Google Colab

- **Code-Completion** und **Code-Generierung** direkt im Notebook
- **Learn Mode:** erklärt Schritt für Schritt und begründet den Lösungsweg, statt Code zum Kopieren zu liefern

Für die Übungen im Kurs: **Learn Mode einschalten**, “Code generieren” nur in Ausnahmefällen.

3. Agenten: die größte Versuchung

- Agenten bearbeiten **ganze Aufgaben**: planen, Code schreiben, ausführen, Fehler korrigieren
- Beispiele: GitHub Copilot Agent Mode (VS Code), Claude Code, OpenAI Codex
- Teilweise schon in **kostenlosen Tarifen** nutzbar
- Für Profis ein Produktivitätswerkzeug – für Anfänger ein Lernverhinderer: Sie sehen ein Ergebnis, aber keinen Lösungsweg

In der Klausur programmieren Sie **ohne KI** – üben Sie so, wie Sie geprüft werden.

3. Empfehlung: Welches Tool wofür?

Zweck	Empfohlen
Übungen im Notebook	Google Colab + **Learn Mode**
Konzepte verstehen	Chat im **Lernmodus** (ChatGPT, Gemini, Claude)
Fehlermeldung verstehen	Chat im Lernmodus, mit vollständiger Fehlermeldung
Code-Completion in der IDE	GitHub Copilot (**erst ab Semester 2**)
Agenten	**nicht in diesem Kurs**

3. Empfehlung: Grundsätze

- Completion und Agenten **erst nutzen, wenn die Grundlagen sitzen**
- KI als **Sparringspartner**, nicht als **Ghostwriter**
- Nutzung in Übungsabgaben: **immer angeben!**

Erst die Konzepte verstehen, dann KI als Werkzeug einsetzen.

Generative KI-Tools: Regeln in diesem Kurs

- Verwenden Sie KI-Tools wie ChatGPT für Ihr Studium
- Kritische Auseinandersetzung mit Ergebnissen zwingend notwendig
- **Angabe** welche Tools, wo und wie eingesetzt wurden
- Nicht-Nennung = **Plagiat**

Die Lösung Ihrer Übungsaufgaben muss eigenständig erfolgen! GenKI-Einsatz zur Erstellung der Lösung ist unzulässig und wird als Plagiat gewertet.

4. Typische KI-Fehler: Die Zahlen

- **62%** der KI-generierten C-Programme haben Sicherheitslücken (Tihanyi et al. 2024)
- **~40%** des Copilot-Codes enthält Bugs oder Schwachstellen (NYU 2021)
- **59%** der Entwickler nutzen KI-Code, den sie nicht verstehen (Clutch.co)
- Nur **29%** vertrauen der Genauigkeit von KI-Output (Stack Overflow 2025)

4. Typische KI-Fehler: Vier Kategorien

1. **Halluzinationen** – nicht-existente APIs, erfundene Funktionen
2. **Falsche Logik** – Code sieht plausibel aus, ist aber falsch
3. **Sicherheitslücken** – unsichere Patterns, fehlende Validierung
4. **Veraltetes Wissen** – deprecated APIs, alte Library-Versionen

KI kann nur **29,2%** der eigenen Bugs nach zwei Iterationen beheben (Dou et al. 2024).

4. Halluzinationen

KI erfindet Dinge, die nicht existieren

- **Nicht-existente APIs** und Funktionssignaturen (CodeMirage, Agarwal et al. 2024)
- 10 dokumentierte Bug-Muster: Misinterpretation, Hallucinated Object, Missing Corner Case u.a. (Tambon et al. 2024)

4. Falsche Logik

Code sieht korrekt aus, ist es aber nicht

- Code, der auf den ersten Blick plausibel aussieht, aber bei Edge Cases versagt
- **EvalPlus** (Liu et al. 2023): Bei erweiterten Tests (80x mehr) sinkt die Pass-Rate um **19–29%**
- Standard-Tests erkennen die Fehler **nicht**
- Modelle sind schlecht darin, **eigene** Fehler zu erkennen

4. Sicherheitslücken

- **62%** der KI-generierten Programme sind verwundbar (Tihanyi et al. 2024, 331.000 Programme)
- **29,5%** der Copilot Python-Snippets haben Sicherheitsschwächen (Fu et al. 2023)
- **Kein Zusammenhang** zwischen “Code funktioniert” und “Code ist sicher” (Sabra et al. 2025)
- Hardcoded Passwords, Path Traversal, fehlende Input-Validierung

4. Veraltetes Wissen

- **48% bessere** Ergebnisse für Probleme **vor** dem Training-Cutoff (Liu et al. 2023)
- Veraltete Library-Versionen, deprecated Funktionen
- KI hat keinen Zugriff auf aktuelle Dokumentation

Wenn eine Library ein Update bekommt, weiß die KI davon erst nach dem nächsten Training.

4. Warum passieren diese Fehler?

Eigenschaft	Compiler	LLM
Arbeitsweise	Regelbasiert, deterministisch	Statistisch, probabilistisch
Korrektheit	Garantiert	Nie garantiert
Trainingsgrundlage	Sprachspezifikation	Internet-Code (inkl. Fehler)
Optimierungsziel	Korrekte Ausführung	Plausibel klingende Antwort

4. Warum passieren diese Fehler? (2)

- LLMs generieren das **wahrscheinlichste nächste Token** – nicht die **korrekte** Antwort
- Trainiert auf öffentlichem Code: Stack Overflow, GitHub – inkl. Bugs und unsicherem Code
- Kein echtes “Verständnis” von Programmlogik oder Sicherheitskonzepten

5. Fehlerreduktion: Checkliste

KI-Code immer reviewen – niemals blind übernehmen

1. **Funktionalität:** Macht der Code was er soll?
2. **Edge Cases:** Leere Eingabe, Null, negative Zahlen?
3. **Sicherheit:** Keine Hardcoded Secrets, keine Injections?
4. **Lesbarkeit:** Jeden Ausdruck verstehen? Wenn nein $\rightarrow$ nachfragen

5. Fehlerreduktion: Recursive Criticism

KI den eigenen Code nochmal reviewen lassen

“Prüfe diesen Code auf Fehler, Sicherheitslücken und Edge Cases.”

- Kann Schwachstellen **um eine Größenordnung** reduzieren (OpenSSF)
- Funktioniert besser als einmaliges Generieren

Quelle: OpenSSF Security Guide

5. Fehlerreduktion: Testen

Testfälle VOR dem KI-Einsatz definieren

- Wie im **Programmierprozess Schritt 2**: Erst von Hand lösen, dann Testfälle ableiten
- KI-Code gegen **eigene** Testfälle prüfen
- **TiCoder** (Fakhoury et al. 2024): +45,97% Korrektheit durch testgetriebene Interaktion

5. Fehlerreduktion: Verification Paradox

KI-Tests für KI-Code haben blinde Flecken

- Wenn KI Code **und** Tests generiert, können beide den **gleichen** Fehler haben
- **89%** der Sicherheitslücken behebbar bei gezieltem Feedback (Liu et al. 2023)

> Tests müssen **eigene** Logik widerspiegeln, nicht KI-Logik.

5. Fehlerreduktion: Erklären lassen

Verstehen prüfen, nicht nur Code übernehmen

“Erkläre mir diesen Code Zeile für Zeile. Warum hast du diese Lösung gewählt und keine andere?”

- Danach **selbst** in eigenen Worten erklären – gelingt das nicht, haben Sie den Code nicht verstanden
- Gegenprobe: den Code an einem eigenen Beispiel **von Hand durchspielen** (Tracing aus E2)

5. Wann NICHT KI einsetzen

METR RCT (2025): Erfahrene Entwickler waren mit KI **+19% langsamer** – obwohl sie glaubten, schneller zu sein.

KI vermeiden bei:

- **Grundlagen lernen:** Erst selbst verstehen
- **Prüfungen und Übungsabgaben:** Eigenständige Leistung
- **Sicherheitskritischem Code:** Immer menschliche Prüfung
- **Wenn man den Output nicht versteht**

5. Das größte Risiko: Erosion kritischen Denkens

Studierende und Akademiker benennen dasselbe Problem

- **Verlust eigenständiger Problemlösefähigkeit** – meistgenanntes Risiko über alle Studien hinweg (Schei et al. 2024, 58 Zit.)
- Studierende, die KI **passiv** nutzen, entwickeln keine eigene Denkstruktur
- **Kognitives Outsourcing** verhindert nachhaltiges Lernen (Li et al. 2025)

> Wer nie selbst debuggt hat, wird auch mit KI kein guter Programmierer.

Zusammenfassung

- **KI und Lernen:** Erklärhilfe $\rightarrow$ positiv, Lösungsgenerator $\rightarrow$ negativ
- **KI als Lernpartner:** Lernmodus an, Kontext geben, im Dialog lernen
- **Tools kennen:** Lernmodi ja, Completion später, Agenten nicht in diesem Kurs
- **KI-Fehler:** 62% unsicherer Code, Halluzinationen, falsche Logik
- **Fehler reduzieren:** Reviewen, testen, erklären lassen
- **Grundlagen zuerst:** Erst Konzepte verstehen, dann KI nutzen

Quellen: Studien (1/2)

- Dell’Acqua et al. (2023): “Navigating the Jagged Frontier” – HBS Working Paper 24-013
- Sunil et al. (2025): “SocraticAI” – arxiv.org/abs/2512.03501
- Gupta et al. (2025): “Sakshm AI” – arxiv.org/html/2503.12479
- Alanazi et al. (2025): Meta-Analyse KI-gestütztes CS-Lernen
- Li et al. (2025): GenKI und langfristiger Wissenstransfer
- Chang et al. (2023): SRL-Designprinzipien, Reverse Prompting (198 Zit.)

Quellen: Studien (2/2)

- Tihanyi et al. (2024): 331.000 KI-generierte Programme
- Fu et al. (2023): Copilot Security Weaknesses
- Tambon et al. (2024): 10 Bug-Patterns in LLM-Code
- Liu et al. (2023): EvalPlus – HumanEval+
- Fakhoury et al. (2024): TiCoder
- METR (2025): metr.org RCT Study
- El Fathi et al. (2025): CILP Framework, gezielter KI-Einsatz
- Schei et al. (2024): Scoping Review KI-Chatbots (58 Zit.)
- Parsakia (2023): Chatbots und Selbstwirksamkeit

Quellen: Surveys, Tools und Guides

- Stack Overflow 2025: survey.stackoverflow.co/2025/ai/
- GitHub Copilot: code.visualstudio.com/docs/copilot
- Colab Learn Mode: blog.google/…/colab-updates
- ChatGPT Study Mode: openai.com/index/chatgpt-study-mode
- Gemini Guided Learning: techcrunch.com (06.08.2025)
- Claude Learning Mode: engadget.com (14.08.2025)
- Claude Code: anthropic.com/product/claude-code
- OpenSSF: best.openssf.org